

データ構造 スパーステーブル Disjoint Sparse Table 区間クエリ モノイド

Disjoint Sparse Table

1. 概要

本講座では、静的な列に対する区間クエリを高速に処理するデータ構造である **Disjoint Sparse Table**（**DST** と略記することもあります）を解説します。

Disjoint Sparse Table は、長さ N の列に対して事前計算 $O(N \log N)$ 時間を行うことで、モノイドに対する区間積クエリをクエリあたり $O(1)$ 時間で処理するデータ構造です。

同様の計算量はスパーステーブルでも達成できましたが、スパーステーブルが適用できるのは二項演算が**べき等**な場合（ $\min$, $\max$, $\gcd$ など）に限られていました。Disjoint Sparse Table はその制約を必要としない形で同様の計算量を実現するデータ構造だと考えることができます。

なお、スパーステーブルと Disjoint Sparse Table とは呼称は似ていますが、アルゴリズムは完全に別のものだと考えて学ぶのがよいと思います。

2. 前提知識

スパーステーブル の理解を前提とします。ただし、スパーステーブルのアルゴリズムの詳細は本講座には必要ありません。

また必須ではありませんが、モノイドについてある程度理解していると、本講座の内容を理解しやすくなります。必要であればモノイドについては 代数構造用語集 を参照してください。

3. 問題設定

まずは抽象代数の用語を用いながら、問題設定を説明します。モノイド等の用語に馴染みがない方は、直接具体例（**【問題 2】**，**【問題 3】**，**【問題 4】**）を確認していただければよいと思います。

す.

【問題 1】

$(M, *)$ をモノイドとします. M の元からなる長さ N の列 $A = (A_0, A_1, \dots, A_{N-1})$ が与えられます. Q 個のクエリに答えてください. クエリでは $0 \leq L < R \leq N$ を満たす整数 L, R が与えられるので,

$$A_L * A_{L+1} * \dots * A_{R-1}$$

を出力してください.

また, 計算量解析に際しては, M の元に対する操作 (配列への読み書きやモノイド演算) が $O(1)$ 時間であるとします.

例えば次のような問題が具体例となります.

【問題 2】

整数列 $A = (A_0, A_1, \dots, A_{N-1})$ が与えられます. Q 個のクエリに答えてください. クエリでは $0 \leq L < R \leq N$ を満たす整数 L, R が与えられるので, 総和 $A_L + A_{L+1} + \dots + A_{R-1}$ を求めてください.

【問題 3】

整数列 $A = (A_0, A_1, \dots, A_{N-1})$ および正整数 m が与えられます. Q 個のクエリに答えてください. クエリでは $0 \leq L < R \leq N$ を満たす整数 L, R が与えられるので, 総積 $A_L A_{L+1} \dots A_{R-1}$ を m で割った余りを求めてください.

【問題 4】

整数列 $A = (A_0, A_1, \dots, A_{N-1})$ が与えられます. Q 個のクエリに答えてください. クエリでは $0 \leq L < R \leq N$ を満たす整数 L, R が与えられるので, 区間最小値 $\min(A_L, A_{L+1}, \dots, A_{R-1})$ を求めてください.

ここで挙げた問題には, 与えられる列が**静的** (static), つまり変更されることがないという特徴があります. このことを活かして, これらの問題をさらに高速に解くことが目標です.

なお、具体例として挙げた問題のうち 【問題 2】 については、累積和を用いると、 $O(N + Q)$ 時間で解くことができます。これは

$$A_L + A_{L+1} + \cdots + A_{R-1} = (A_0 + A_1 + \cdots + A_{R-1}) - (A_0 + A_1 + \cdots + A_{L-1})$$

を利用するものです。このような解法は 【問題 1】、【問題 3】、【問題 4】 ではできないことに注意してください。 $[0, R)$ と $[0, L)$ の場合から $[L, R)$ の場合の答を求めるための「引き算」のような計算が行えないからです。代数学の言葉でいえば、累積和を用いた解法はモノイドが逆元を持つ（群である）場合などに限られます。

4. Disjoint Sparse Table

以下では、Disjoint Sparse Table を **DST** と略記します。

4.1. 分割統治の考え方

Disjoint Sparse Table は、列の**分割統治法**（divide and conquer）の考え方で 【問題 1】 を解くデータ構造です。

「列の分割統治」といった場合に指す内容は多義的ですが、ここでは、区間 $[a, b)$ に含まれる区間 $[L_i, R_i)$ ($i \in I$) に対して何らかの処理をするという問題を、次の手順で解く方法を指します。

- $b = a + 1$ ，つまり $[a, b)$ の長さが 1 の場合には何らかの方法で解く。以下 $b - a \geq 2$ であるとする。
- $c = \left\lfloor \frac{a+b}{2} \right\rfloor$ とする。区間 $[a, b)$ は $[a, c)$ と $[c, b)$ に分割される。
- 各区間 $[L_i, R_i)$ を次の 3 種類に分類する。
 - 左側に含まれるもの。つまり $[L_i, R_i) \subseteq [a, c)$ であるようなもの。
 - 右側に含まれるもの。つまり $[L_i, R_i) \subseteq [c, b)$ であるようなもの。
 - 境界 c をまたぐもの。つまり $L_i < c < R_i$ が成り立つもの。
- 左側に含まれるものに対する問題を、再帰的に解く。
- 右側に含まれるものに対する問題を、再帰的に解く。
- 境界 c をまたぐものに対する問題を、何らかの方法で解く。

▶ 擬似コード

このような分割統治法を用いる場合、多くの場合には $b = a + 1$ の場合の処理は簡単です。問題となるのは「境界 c をまたぐもの」に対する処理です。

モノイドの区間積クエリの場合にこの部分をどのように処理するかを見ていきましょう。

4.2. 境界 c をまたぐ場合の処理

$[a, b)$ を長さ 2 以上の区間、 $c = \left\lfloor \frac{a+b}{2} \right\rfloor$ とし、 $[L, R) \subseteq [a, b)$ が境界 c をまたぐ、つまり $L < c < R$ が成り立つとします。

このとき、 $[L, R)$ を境界 c によって、 $[L, c)$ と $[c, R)$ に分割することができます。したがって 【問題 1】 の状況において

$$\prod_{L \leq i < R} A_i = \left(\prod_{L \leq i < c} A_i \right) * \left(\prod_{c \leq i < R} A_i \right)$$

が成り立ちます。（ただし総積 $\prod$ はモノイドの二項演算 $*$ に関する積とし、積の順序は添字について昇順であるものとします。）

したがって、 $[a, c)$ の suffix および $[c, b)$ の prefix について、区間積を求めておくことによって、 $[L, R)$ に対する区間積を計算することができます。

なお、ここまでの考え方を踏まえると、クエリが**オフライン**（offline）である場合、つまり答えるべきクエリがすべてまとめて与えられる場合には、【問題 1】 を解くことができます。

4.3. DST の事前計算（ $N = 2^K$ の場合）

4.1 節、4.2 節の考え方をもとに、区間積クエリに**オンライン**（online）に答えられるようにした（つまり、クエリが与えられるたびに答を求められるようにした）データ構造が **DST** です。

簡単のためしばらく、 $N = 2^K$ が 2 のべき乗であることを仮定します。この場合には分割統治法で登場する区間 $[a, b)$ の長さはすべて 2 のべき乗になります。

そのような区間のうちで長さ 2 以上のもの $[a, b)$ について, 4.2 節で解説したように, 境界 $c = \left\lfloor \frac{a+b}{2} \right\rfloor$ をとり, $[a, c)$ の suffix および $[c, b)$ の prefix に関する区間積を事前計算します.

例えば $N = 8$ としたとき, DST の事前計算で計算される区間積を図示すると次のようになります.

	0	1	2	3	4	5	6	7
$k = 2$	[0, 4)	[1, 4)	[2, 4)	[3, 4)	[4, 5)	[4, 6)	[4, 7)	[4, 8)
$k = 1$	[0, 2)	[1, 2)	[2, 3)	[2, 4)	[4, 6)	[5, 6)	[6, 7)	[6, 8)
$k = 0$	[0, 1)	[1, 2)	[2, 3)	[3, 4)	[4, 5)	[5, 6)	[6, 7)	[7, 8)

各 $k = 0, 1, 2, \dots, K - 1$ について, $[0, N)$ は長さ 2^{k+1} の区間 $[a, b)$ に分割されます. したがって長さ 2^{k+1} の区間に対して計算しておく区間積の個数はちょうど N になり, 事前計算した区間積を $K \times N$ のテーブルとして保持することができます.

形式的には $0 \leq k \leq K - 1$ および $0 \leq i < N$ に対して, 次のようにして定まる $\text{table}[k][i]$ を事前計算します.

- a を N 未満の 2^{k+1} の倍数とする.
- $b = a + 2^{k+1}$, $c = (a + b)/2$ とする.
- $a \leq i < c$ に対して $\text{table}[k][i] = \prod_{i \leq j < c} A_j$ とする.
- $c \leq i < b$ に対して $\text{table}[k][i] = \prod_{c \leq j \leq i} A_j$ とする.

このような計算は, $i = c - 1, c - 2, \dots, a$ の順および $i = c, c + 1, \dots, b - 1$ の順に計算することで, (k, i) ごとに $O(1)$ 時間, 全体で $O(N \log N)$ 時間で行えます.

4.4. DST のクエリ処理 ($N = 2^K$ の場合)

クエリで与えられる区間 $[L, R)$ の長さが 1 である場合には, 単に A_L が答えです. 以下では $R - L \geq 2$ が成り立つとします.

この場合には、 $\prod_{L \leq i < R} A_i$ は、事前計算された値のうち適切な 2 つの値の積として求まることとなります。具体的には次のようになります。

- $\left\lfloor \frac{L}{2^k} \right\rfloor \neq \left\lfloor \frac{R-1}{2^k} \right\rfloor$ かつ $\left\lfloor \frac{L}{2^{k+1}} \right\rfloor = \left\lfloor \frac{R-1}{2^{k+1}} \right\rfloor$ を満たすような非負整数 k を求める。
- $\text{table}[k][L] * \text{table}[k][R-1]$ を出力する。

前者の条件を満たす k は、 L と $R-1$ のビット単位 XOR 演算 $L \oplus (R-1)$ の最上位セットビットインデックスと言い換えることができます。最上位セットビットインデックスやその計算方法については、[スパーステーブル](#) 4.3 節を参照してください。

最上位セットビットインデックスを適切に求めることで、クエリに $O(1)$ 時間で答えることができます。

4.5. N が 2 のべき乗とは限らない場合

これまでは $N = 2^K$ が 2 のべき乗であることを仮定していました。しかし、 N が 2 のべき乗でない場合にもほとんど考え方は変わりません。単に 4.3 節で述べた事前計算を、区間 $[a, b)$ における境界 c の位置が N 未満である部分について行えば十分です。例えば $N = 5$ の場合には、次の図のような事前計算を行えばよいです。

	0	1	2	3	4
$k = 2$	[0, 4)	[1, 4)	[2, 4)	[3, 4)	[4, 5)
$k = 1$	[0, 2)	[1, 2)	[2, 3)	[2, 4)	
$k = 0$	[0, 1)	[1, 2)	[2, 3)	[3, 4)	

なお、この場合には、4.1 節で説明した「常に中央で二分する」分割統治をそのまま実装しているわけではありません。実装では、長さが 2 のべき乗である区間を基準に事前計算を行うため、例えば $[0, 5)$ については $[0, 2)$, $[2, 5)$ ではなく、 $[0, 4)$, $[4, 5)$ への分割を考えています。

4.6. 結論

結論を定理として述べておきます.

【定理 5】

モノイドの区間積クエリは, DST を用いることで $O(N \log N)$ 時間の事前計算のもと, クエリごとに $O(1)$ 時間で処理できる.

したがって **【問題 1】** は $O(N \log N + Q)$ 時間で解くことができます.

4.7. 実装例

Library Checker Static RMQ への提出です.

- <https://judge.yosupo.jp/submission/358326>

5. 関連問題

1. <https://judge.yosupo.jp/problem/staticrmq>
2. <https://www.codechef.com/NOV17/problems/SEGPROD>
3. https://atcoder.jp/contests/abc426/tasks/abc426_g

2 は, N に対して Q が非常に大きいという問題で, セグメント木よりも DST に利点があるタイプの問題設定です.

3 は 4.1 節の分割統治法の考え方が利用できる問題です. DST を用いた解法もありますが, 空間計算量を制約に収めるのが難しいです.

6. まとめ

本講座では, 静的な列に対する区間クエリを高速に処理するデータ構造である **Disjoint Sparse Table** を解説しました. 結論として, モノイドの区間積クエリは, DST を用いることで $O(N \log N)$ 時間の事前計算のもと, クエリごとに $O(1)$ 時間で処理できることが分かりました.

スパーステーブルが、任意の区間を事前計算された 2 つの区間の**和集合**として表すものだったのに対して、Disjoint Sparse Table は任意の区間を事前計算された 2 つの区間の**非交和** (disjoint union) として表すものです。このことにより、スパーステーブルの適用のために必要だったべき等性の制約を仮定することなく、モノイドの区間積に答えることができます。

また、Disjoint Sparse Table に使われた、分割統治法における

- $[a, c)$ の suffix および $[c, b)$ の prefix について適切な事前計算を行い、
- それらを結合することで、境界 c をまたぐ場合の処理を行う

というアイデアも、多くの場面で用いられる重要なものです。

7. 参考文献

1. data-structures. Disjoint Sparse Table. https://scrapbox.io/data-structures/Disjoint_Sparse_Table. (閲覧日: 2026-03-09).
2. nilesh3105. [Tutorial] Disjoint Sparse Table. CodeChef Discuss. <https://discuss.codechef.com/t/tutorial-disjoint-sparse-table/17404>. (閲覧日: 2026-03-09).
3. noshi91. Disjoint Sparse Table と セグ木に関するポエム. <https://noshi91.hatenablog.com/entry/2018/05/08/183946>. (閲覧日: 2026-03-09).
4. noshi91. Disjoint Sparse Table に乗るのはやはりモノイドかもしれない. <https://noshi91.hatenablog.com/entry/2023/04/07/165310>. (閲覧日: 2026-03-09).
5. maspyのHP. 分割統治による静的列の区間積クエリ. <https://maspy.py.com/分割統治による静的列の区間積クエリ>. (閲覧日: 2026-03-09).

トップページ : [AtCoder Algorithm Lectures](#)

質問・誤植報告・補足情報など : [Discord サーバー](#)



AtCoderは、世界最高峰の競技プログラミングサイトです。 リアルタイムのオンラインコンテストで競い合うことや、 5,000以上の過去問にいつでもチャレンジすることができます。



[ライセンス表記](#)

AtCoderについて

[AtCoderとは？](#)

[競技プログラミングとは？](#)

[レーティングのしくみ](#)

[アクセスガイド](#)

[お知らせ](#)

資料請求

[AtCoder](#)

[AtCoderJobs](#)

各サービス

[AtCoder](#)

[AtCoderJobs](#)

[アルゴリズム実技検定](#)

[AtCoderCareerDesign](#)

AtCoder株式会社について

[会社概要](#)

[FAQ](#)

[プライバシーポリシー](#)

[利用規約](#)